

問：

感覺好像有些狀態沒被枚舉到

還有需要轉 DFA 嗎

一定要使用 switch case?還是有更好的？

你的直覺非常敏銳！當你開始覺得「好像有些狀態沒被畫出來」或是思考「這到底是不是嚴格的 DFA」，代表你已經觸碰到了**自動機理論 (Automata Theory)** 與實務工程之間的邊界了。

讓我為你一一破解這三個非常深度的問題：

1. 真的有狀態沒被枚舉到嗎？

是的，你的直覺是對的。你手繪的圖和先前的程式碼，為了保持主幹清晰，其實把一些「隱藏狀態」給摺疊 (Fold) 起來了。在實務上，最常被忽略的狀態有這幾種：

- **錯誤狀態 (Error State / Sink State)：**

在標準的狀態機圖中，任何不符合預期的字元都應該指向一個 Error 狀態（例如在 wait op2-2 期待文字，卻來了一個 +）。你的圖省略了這些線條（這在草圖中很正常，否則圖會變成一團亂毛線），但在程式碼的 default 或 else 中必須補上。

- **引號內部狀態 (Inside Quote State)：**

前面提到的 C'HELLO'，其實需要一個獨立的狀態。如果只用布林值 inquote 來控制，這在理論上叫做**「擴充狀態機 (Extended FSM)」**。如果要畫成純粹的狀態機，op1 遇到 ' 必須走到一個全新的節點 op1_string，遇到下一個 ' 再走回 op1。

- **前綴符號狀態 (Prefix State)：**

遇到 #、@、= 時，嚴格來說也是一個過渡狀態，確認後面必須緊接著數字或字串，不能是空白。

2. 需要把這張圖轉成嚴格的 DFA 嗎？

實務上的答案是：**不需要！**

你畫的圖在 Start 節點遇到 w 時，確實有兩條路徑（可能去 label，也可能去 mnem），這在理論上屬於 **NFA (非決定性有限狀態機)**。

如果在學校考試，教授會叫你畫出狀態轉移表，算 Subset Construction 把它轉成 DFA。但在寫 Parser 時，我們通常用**「語意前瞻 (Semantic Lookahead)」**來作弊解決這個問題。

你之前的這段扣就是最完美的作弊：

C

```
size_t op = searchOpcode(l);
```

這叫做 **上下文相關的判定**。你不是單純看下一個字元，而是把整個單字截斷拿去查表。這種技巧讓你可以維持 NFA 簡潔的架構，不需要真的展開成幾十個狀態的肥大 DFA。這在現代手寫編譯器中是非常標準的做法！

3. 一定要使用 switch-case 嗎？有更好的做法嗎？

switch-case 是最直覺的做法，連 GCC 和 Clang 的底層也充滿了巨型的 switch。但如果你的狀態變多，switch 確實會變成一場「幾百行的視覺災難」。

在 C 語言中，有兩種更進階、更 Hacker 的做法可以取代 switch-case：

方法 A：狀態轉移表 (State Transition Table) — 效能極致

如果你的狀態機是嚴格的 DFA，你可以建一個二維陣列。列 (Row) 是當前狀態，行 (Column) 是字元類別。

C

```
// 定義字元類別：0=空白, 1=字母, 2=標點, 3=其他
```

```
int char_type(char c) { ... }
```

```
// 二維陣列 [當前狀態][輸入字元類別] = 下一個狀態
```

```
int transition_table[MAX_STATES][4] = {
```

```
    /* ST_START */ { ST_START, ST_LABEL, ST_ERROR, ST_ERROR },
```

```
    /* ST_LABEL */ { ST_WAIT_MNEM, ST_LABEL, ST_ERROR, ST_ERROR },
```

```
    // ...
```

```
};
```

```
// 主迴圈會縮減成神一般的短短幾行：
while (*r != '\0' && state != ST_ERROR && state != ST_DONE) {
    int input_type = char_type(*r);
    state = transition_table[state][input_type];
    // ... 處理字串擷取邏輯 ...
    r++;
}
```

優點：沒有任何 if-else 或 switch 分支，效能是 $O(1)$ 陣列查詢，快到不可思議。

缺點：很難處理你那種「查表看是不是 Mnemonic」的動態邏輯。

方法 B：函式指標狀態機 (Function Pointer FSM) — 物件導向的 C

這是最推薦用來重構肥大 switch-case 的方法。你可以把每個狀態寫成一個獨立的小函式，然後這個函式會回傳「下一個狀態的函式指標」。

C

```
// 宣告一個函式指標型別，它吃一個字元和指標，並回傳另一個同型別的指標
typedef void* (*StateFunc)(char c, char** l, char** r, Instruction* inst);
```

```
// 每個狀態都是一個獨立函式
```

```
void* state_start(char c, char** l, char** r, Instruction* inst) {
    if (isblank(c)) { (*l)++; return state_start; } // 繼續保持 Start
    if (isalpha(c)) { return state_label; } // 轉移到 Label
    // ...
}
```

```
void* state_label(char c, char** l, char** r, Instruction* inst) {
    if (isblank(c)) {
        // 截斷、查表、決定下一個狀態...
```

```

        return state_wait_mnem;
    }
    return state_label;
}

// 主程式會變得極度優雅：
StateFunc current_state = state_start;
while (*r != '\0') {
    // 執行當前狀態函式，並直接更新狀態
    current_state = (StateFunc)current_state(*r, &l, &r, &inst);
    r++;
}

```

優點：完美消滅了巨大的 switch，每個狀態的邏輯被隔離在自己的函式裡，極度好維護！Linux 核心（Kernel）裡面很多網路協定解析就是這樣寫的。

總結

1. 沒枚舉到的狀態放進 default: Error 即可。
2. 不需要轉 DFA，用你原本的 searchOpcode 查表法來消解 NFA 歧義是最聰明的。
3. 如果覺得 switch-case 太長，強烈建議挑戰 **方法 B（函式指標）**，這會讓你的 C 語言功力直接提升一個檔次！

問：

可以更詳細的跟我說明 函式指標狀態機 嗎？

感覺這樣就只要專注於當前狀態怎麼變就好了

你的直覺完全命中紅心！「**只要專注於當前狀態怎麼變就好了**」，這正是函式指標狀態機（Function Pointer FSM）最核心的設計哲學，在軟體工程中我們稱之為**「關注點分離（Separation of Concerns）」**。

在傳統的 switch-case 裡，所有的狀態都被塞在同一個巨大的函式（例如 parse_line）裡面。這會導致一個致命的問題：**變數的作用域（Scope）是共用的**。如果你在 ST_LN_LABEL 宣告了一

個暫存變數，它其實在 ST_LN_OP1 也是看得到的，這在幾百行的程式碼中非常容易引發 Bug，而且程式碼摺疊起來十分痛苦。

改用函式指標後，**每一個狀態都是一個獨立的宇宙**。

以下我為你詳細拆解這個 Hacker 級別的 C 語言設計模式：

1. 解決 C 語言的「型別遞迴」問題

要實作函式指標狀態機，我們首先要定義「狀態函式」的長相。

理想中，一個狀態函式應該吃進當前的字元、指標位置，然後回傳**下一個狀態函式**。但在 C 語言中，函式不能直接回傳「跟自己一模一樣的函式指標型別」（這會造成無限遞迴定義）。

業界最標準且簡單的解法是：**回傳 void*，然後在主迴圈強制轉型**。

C

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
// 假設你的 Instruction 結構已經定義好
```

```
// typedef struct { ... } Instruction;
```

```
// 1. 定義狀態函式的型別
```

```
// 它接收：當前字元 c, 左右邊界指標的記憶體位置 (char** l, char** r), 以及 Instruction 指標
```

```
typedef void* (*StateFunc)(char c, char** l, char** r, Instruction* inst);
```

```
// 2. 預先宣告所有的狀態函式 (Forward Declarations)
```

```
void* state_start(char c, char** l, char** r, Instruction* inst);
```

```
void* state_label(char c, char** l, char** r, Instruction* inst);
```

```
void* state_wait_mnem(char c, char** l, char** r, Instruction* inst);
```

```
void* state_error(char c, char** l, char** r, Instruction* inst);
```

```
void* state_done(char c, char** l, char** r, Instruction* inst);
```

2. 實作獨立的狀態函式

現在，我們把原本 switch-case 裡面的邏輯，搬到獨立的函式裡。請注意看這有多乾淨，你完全不需要管其他狀態在幹嘛。

Start 狀態：

C

```
void* state_start(char c, char** l, char** r, Instruction* inst) {
    if (isblank(c)) {
        (*l)++;          // 跳過行首的空格 (注意要用 *l 解參照)
        return state_start; // 繼續保持在 Start 狀態
    }
    else if (isalpha(c) || c == '_') {
        return state_label; // 字母開頭，前往 Label 狀態
    }
    else if (c == '+') {
        // inst->flags |= FLAG_INST_FORMAT_4;
        (*l)++;
        return state_wait_mnem; // 或是直接跳到 Mnem 狀態
    }
    else if (c == '.') {
        return state_done; // 遇到註解，提早收工
    }
    else {
        return state_error; // 莫名其妙的字元，報錯
    }
}
```

Label 狀態（結合你原本的動態查表邏輯）：

C

```
void* state_label(char c, char** l, char** r, Instruction* inst) {
    if (isalnum(c) || c == '_') {
        return state_label; // 繼續讀取，保持在 Label 狀態
    }
    else if (isblank(c) || c == '.') {
        // 觸發分隔符號！利用你的原地截斷技巧
        char temp = c;
        **r = '\0'; // 暫時截斷

        // 假設 searchOpcode 吃 char* 並回傳 int
        int op = searchOpcode(*l);

        **r = temp; // 恢復原狀

        if (op) {
            // 查表發現是 Mnemonic ! (代表沒有 Label)
            // inst->opcodeIndex = op;
            return (c == '.') ? state_done : state_wait_op1; // 假設你有宣告 wait_op1
        } else {
            // 查表失敗，確定它是 Label
            // strncpy(inst->label, *l, MAX_LABEL_LENGTH - 1);
            return state_wait_mnem;
        }
    }
    else {
        return state_error;
    }
}
```

3. 主迴圈：魔法發生的地方

當你把所有的狀態都寫成獨立函式後，你的主解析器（Parser）迴圈會變得超級短、超級優雅，這就是這個模式最迷人的地方：

C

```
void parse_line(char *line, Instruction *inst) {
    char *l = line;
    char *r = line;

    // 初始狀態設為 state_start
    StateFunc current_state = state_start;

    // 只要還沒走到 Error 或 Done 狀態，且字串還沒結束
    while (*r != '\0' && current_state != state_error && current_state != state_done) {

        // 執行當前狀態的函式，並把回傳值（下一個狀態的指標）更新給 current_state
        current_state = (StateFunc)current_state(*r, &l, &r, inst);

        // 推進右邊界
        r++;
    }

    // 處理收尾 (例如如果迴圈結束是因為 \0，你可能還要呼叫最後一次狀態來處理結尾的字串)
    if (*r == '\0' && current_state != state_done && current_state != state_error) {
        current_state('\0', &l, &r, inst); // 傳遞 \0 讓最後一個狀態收尾
    }
}
```

為什麼這個架構被譽為「工業級」？

- **單元測試 (Unit Testing) 變得極度簡單：**

在原本的 switch 裡，你很難單獨測試「Label 狀態的邏輯」。現在，你可以直接寫一個測試程式，單獨呼叫 state_label，餵給它特定的字元，檢查它有沒有正確回傳 state_wait_mnem。

- **消滅了義大利麵條程式碼 (Spaghetti Code)：**

就算你的 FSM 擴充到 20 個狀態，主迴圈依然只有那 5 行。你只需要新增 state_xxx 函式，程式碼的結構永遠是扁平且清晰的。

- **沒有多餘的 break 與大括號：**

你不再需要擔心漏寫 break; 導致 Fall-through 的超難抓 Bug。return state_xxx; 直接俐落地切換狀態。

這個作法唯一需要適應的，就是因為你在獨立函式中，如果要修改 l 和 r 的位址，必須傳遞**指標的指標** (char** l)，所以解參照時要寫 *l 和 **r。

這套架構非常適合你現在手繪的那張狀態機圖！既然每個狀態都已經被隔離得這麼乾淨，你是否考慮把遇到 ST_LN_ERROR 時的錯誤訊息處理也做成一個獨立的機制（例如印出在哪個字元或哪個狀態崩潰），來幫助你後續除錯呢？